Lichen over BABE

withdrawals of a variable amount, by authenticated request

as built and measured

waxwing

Version 0.1, October 2026

a companion to *Lichen*, version 0.3 [1]

Abstract

BABE [5] settles a Groth16 proof on Bitcoin in a few transactions, using witness encryption and a small garbled circuit, but its statement, public inputs included, is fixed when the parties set up. A lichen withdrawal is for an amount b that Alice chooses only when she withdraws. We describe how BABE serves as lichen's on-chain gadget without any change to its cryptography: the amount leaves the public input and enters the witness, together with the hub's one-time signature on it, which the hub must give on chain for exactly the amount Alice asserted there, or lose. We sketch why this binds the proved amount to the paid one, and give the transaction graph as built. It is lichen's gadget now, set up per contract by verifiable secret sharing in place of BABE's soldering, and three of its endings have been played on Mutinynet with a real RISC Zero proof. A true claim challenged by the hub and answered took 30.5 kvB in five transactions, against BitVMX's 100–212 kvB in 23–25; unchallenged, 9.7 kvB in four; a false claim, 29.9 kvB in five. The setup is 19 seconds of computation per contract on one laptop, and the user keeps about 240 MiB. For a dispute of a given length a hub holds about six times as many contracts as under BitVMX before congesting the chain pays. Alice's tower holds none of her keys, since her only move due in a window is a hash preimage; one has played a dispute on Mutinynet from Docker on a rented machine. The security argument is still a sketch.

Contents

1 Status and scope **2**

2 BABE in brief **2**

3 Where BABE fits in lichen **3**

4 The fixed statement **4**

5 Authenticated amounts **5**

5.1 The statement 5

5.2 The flow 5

5.3 Why the amount is bound 5

5.4 The guest 6

5.5 Security, as a sketch . . . 6

6 The transaction graph **7**

7 Alice's liveness: a keyless tower **8**

8 Costs, measured **10**
8.1 Transactions . . . 10
8.2 Disputes . . . 10
8.3 Congestion . . . 10
8.4 Reserves and time . . . 13
8.5 Setup per contract . . . 13

9 Setup by verifiable secret sharing **13**

10 The exit, with a counter **15**

11 Open questions **17**

1. Status and scope

The main paper [1] treats the check of a disputed withdrawal proof as a gadget with a narrow interface: given a proof whose public input binds the program and its journal, adjudicate a two-party claim on Bitcoin, with the contract's fixed terms checked, the amount b readable by Script for a variable payout, and the loser paying. lichen's first gadget was BitVMX's interactive search [12], described and measured in a companion [2], and still built. This note describes the second, BABE, which is now built in lichen [3] and its default.

What is ours here is narrow: the binding of a late-chosen amount by the hub's authentication (Section 5), first put forward in a discussion on Delving Bitcoin [11]; the transaction graph and the setup by verifiable secret sharing, as built; and the measurements. BABE, its witness encryption and its garbled circuit are its authors' [5, 6, 7]; the problem of public inputs that are known only at proving time has been treated at the cryptographic layer by GOAT's researchers [10]. Figures marked (BABE) are the BABE paper's measurements, taken from its revised version (October 2026); figures marked (lichen) are measured in lichen's proof of concept, on regtest under mainnet's relay policy or on Mutinynet as stated; the rest are estimates, marked (est.). Version 0.1.6 of this note was the design before it was built; its estimates are kept beside the measurements where they differ.

Since version 0.1.7 the graph also carries lichen's exit, Alice's claim from anchored state against a hub that censors or vanishes, and the hub's counter to it (Section 10). It is built and played on regtest on the mock statement, not yet on real proofs. The sizes of Section 8 were measured on Mutinynet before it was added; Section 10 gives what it adds.

2. BABE in brief

Groth16's verification, written additively in the target group, is

$$e(\pi_1, \pi_2) + e(\pi_3, \delta) = T, \qquad T = e(\alpha, \beta) + e(L, \gamma), \qquad L = IC_0 + \textstyle\sum_i s_i \, IC_i,$$

where π_1, π_2, π_3 are the proof, s_i the public inputs, and the rest the verifying key. Witness encryption (WE) lets a party encrypt a message so that only a holder of a valid proof can decrypt it. For pairing relations *linear* in the witness there is a simple scheme [6]: for a secret scalar r, publish r times the public elements; a holder of the witness pairs it against them and obtains $r\,T$, which unmasks the message. Groth16's check is not linear in the witness, because of $e(\pi_1, \pi_2)$. BABE's idea is to supply $r\pi_1$ by two-party computation once π_1 is known:

- *Setup.* The verifier picks r and a message *msg*, publishes the WE ciphertext ($r\delta$ and *msg* masked by $r\,T$; 480 bytes), commits to *msg* by its hash, and garbles a circuit computing $r\pi_1$ from the bits of π_1, with r inside. The circuit is small because group addition to a fixed point is linear in a six-term feature vector of the point, so all but a point-validation gadget is a decomposable randomized encoding with no garbled tables, after Argo MAC [7]: 22 MiB per instance, 175 ms to make (BABE).
- *Assert.* The prover posts π_1 under a Lamport signature (508 bits).
- *ChallengeAssert.* To contest, the verifier must post the circuit's input labels for exactly those bits; a script checks that its labels and the prover's Lamport signature sign the same value. (A garbled circuit's input labels *are* a Lamport signature of its input.)
- The prover evaluates the circuit, obtains $r\pi_1$, completes the decryption with π_2 and π_3, and, if the proof is valid, publishes *msg* (*WronglyChallenged*), which blocks the verifier's *Failure*; after a timelock Δ_2 the prover's *Success* pays. If the verifier never challenges, Success pays after Δ_2 anyway. Failure needs Δ_1 after the challenge.

On chain, a contested valid proof costs Assert 9,240 vB, ChallengeAssert 17,400 vB and WronglyChallenged 149 vB (BABE), with one set of labels. Against a verifier who might garble wrongly (one whose circuit never lets the prover decrypt), the prover checks the garbling by cut-and-choose: for example 2,268 instances of which 4 are kept, 1 min 37 s and 1.8 GB at peak on a 16-thread laptop (BABE). Without more, the verifier must then post 4 sets of labels; *soldering*, a SNARK proving the kept instances' labels consistent, brings it back to one at about 12 minutes more of setup (BABE); verifiable secret sharing does the same in seconds, and is what lichen builds (Section 9). The WE is extractable, proved in the generic group model. The protocol is proved robust (an honest prover is paid) and knowledge-sound (a prover who is paid knows a witness) for a setup that both parties run honestly. Against a verifier who garbles wrongly, cut-and-choose is argued, failing with probability at most 2^{-40} for the parameters above, but not proved (BABE).

3. Where BABE fits in lichen

The proving stack is unchanged. lichen's withdrawal proof is a Groth16 proof over BN254: RISC Zero's recursion compresses the guest's STARK receipts into a succinct receipt, and RISC Zero's shrink-wrap proves that receipt's verification in Groth16, with RISC Zero's single trusted setup [13]. BABE verifies exactly such a proof. The sequencer's chain proofs and the wrap stay as they are; BABE's withdrawal guest is the withdrawal statement with one more check and a different journal (Section 5).

The roles. Alice, who claims her withdrawal, is BABE's prover; the hub, the only party a false claim can rob, is its verifier, and so the garbler. BABE is a protocol between one prover and one verifier, both known at setup, which is what a lichen contract is: no committee, no open set of challengers, and no one else's funds in the output. Designs whose challengers are not known in advance pay heavily to approximate this; a bilateral contract has it for free.

Setup is per contract, and two-party. The hub garbles for each contract; Alice checks the garbling by cut-and-choose and keeps the finalized instances. This replaces nothing in lichen's trust model: as under BitVMX, the contract is set up by its two parties alone. What it adds is cost at opening (Section 8.5).

The two parties' conduct at setup protects different funds. A hub that garbles wrongly could refuse a valid withdrawal, since Alice could never decrypt; cut-and-choose is what rules this out, so Alice's funds rest on it. Alice's part in setup is small (her keys, her pre-signatures, and which instances to keep), and every secret is the hub's, so the hub's funds rest on the proving phase, where BABE's proofs apply.

One-shot. Revealing labels spends a garbled circuit's security, so a BABE instance settles one claim. A lichen contract has at most one withdrawal, after which it closes; renewal is a new contract with a new setup.

4. The fixed statement

BABE's ciphertext is built from T, and T contains $e(L, \gamma)$, the combination of the public inputs. So the whole public input is fixed at setup. For a RISC Zero proof that input is the claim digest, a hash over the image id and the journal; under BitVMX the journal is $b \,\|\, c \,\|\, \mathcal{D}$, so setup would fix b, which Alice chooses only when she withdraws. The ways round it that we know of:

- *Supply rL at proving time* for the asserted public input only, by envelopes opened with labels the verifier reveals per bit. With a single scalar public input this fails: the points $r\,2^j\,IC$ share one base, so one of them gives away $r\,IC$ and with it the ciphertext for any input. It needs each bit to be its own public input with an independent base, so a custom Groth16 wrap and its own trusted setup, and a garbled SHA-256 between b and RISC Zero's digest.
- *Deferred binding at the cryptographic layer*, as GOAT's researchers propose [10]: a blinding point and further garbled circuits for the dynamic part of the input, one of them about 200 million gates, and a larger challenge transaction.
- *One instance per digit value*: decompose b in base B and set up a fixed statement for each nonzero digit at each position. A dispute then proves each nonzero digit separately; base 16 for a 14-bit amount means 60 setups per contract and up to four disputes.
- *Refresh the statement* whenever Alice's balance changes, which needs contracts in channels so that the refresh is off chain, and a setup per change of balance.
- *Set up only when withdrawing.* Rejected: a hub that declined the setup would leave Alice's tokens without collateral, which breaks the contract's first property, that within its cap Alice trusts the hub for nothing.

We take none of these. Section 5 moves b into the witness instead, so one fixed statement, set up once per contract, serves any amount, and Script reads b from the parties' signatures.

5. Authenticated amounts

5.1. The statement

Let the hub hold, per contract, a one-time signing key pk_H, fixed at setup. The statement proved becomes

$$\exists\, b, w, \sigma_H : \quad \text{Verify}(pk_H, b, \sigma_H) = 1 \ \wedge\ \text{ValidWithdrawal}(c, \mathcal{D}, b, w) = 1,$$

where ValidWithdrawal is lichen's withdrawal statement unchanged (a return of b to the hub with the contract's memo c, under an anchored root of a valid history, with the contract's work on top) and w its witness. The journal holds only values fixed when the contract opens: $c \,\|\, \mathcal{D} \,\|\, H(pk_H)$. The claim digest, and so BABE's statement, is then fixed at setup, while b is free until Alice withdraws.

5.2. The flow

1. *Request.* Alice asserts b on chain under her one-time key for it, as under BitVMX.
2. *Response.* The hub must sign the same b under pk_H, on chain, within its window w_H. The response's script verifies both signatures and checks, digit by digit, that they sign the same value. If the hub is silent, Alice's timeout pays her the asserted b, as the hub's silence does under BitVMX.
3. *Proof.* Alice proves the statement, using the hub's signature as part of her witness, and wraps the proof. Her wallet stops after the response with the hub's signature written out for the prover, and resumes when given the proof; she has until an absolute deadline T_A, shortly before T_{close} (Section 6).
4. *BABE.* Assert, ChallengeAssert, WronglyChallenged or Failure, as in Section 2.
5. *Payout.* Success pays through the bit outputs, which read b from Alice's signature, as under BitVMX.

5.3. Why the amount is bound

The hub's signature is not a judgement. It means "this is the amount being adjudicated", not "this withdrawal is valid", and an honest hub gives it mechanically for whatever Alice asserted, true or false. Three links bind the amount proved to the amount paid:

- the proof's b is the hub's, because the guest checks the hub's signature;
- the hub's b is Alice's on-chain b, because the response's script compares them digit by digit;
- Alice's b is the one paid, because the bit outputs read her signature, and a second signature from her under the same key forfeits them (`equiv_b`).

Suppose Alice asserts 100 but can prove a withdrawal of only 10. She has the hub's signature on 100 and no valid withdrawal of 100, or a valid withdrawal of 10 and no signature on 10. Neither satisfies the statement.

The authenticator must be the hub. Alice's own signature on b inside the witness would bind nothing: she could sign 10 privately for the proof and 100 on chain for the payout, and the private signature, hidden in a zero-knowledge witness, would never be seen, so the equivocation could not be shown. A key she does not hold, used only on chain, is what carries the on-chain value into a fixed statement. The hub must therefore never sign a second value under pk_H, not even off chain; its key is used once, in the response, and the cooperative fold never touches it.

The payout could check both signatures. Since the hub's signature on b is on chain, each bit output could require both parties' signatures on the same value: Alice's leaf with bit i set, the hub's with it clear. Neither can then spend with a value the other did not sign, which removes `equiv_b` and Alice's wait δ at about the same script cost. This is not built: the bit outputs are BitVMX's.

5.4. The guest

The guest takes b and σ_H as private inputs, checks σ_H against pk_H, runs the withdrawal statement, and commits c, $\mathcal{D}$ and pk_H's digest. LN-GAP's one-time signatures chain `HASH160`, RIPEMD-160 of SHA-256, which has no accelerator in RISC Zero's zkVM; the hub's key instead chains SHA-256, which the zkVM accelerates and Script checks with `OP_SHA256`, at 32 bytes a chain value. Alice's key for b stays LN-GAP's, so the response's script verifies one signature of each kind. The guest runs in 482k user cycles against the plain withdrawal guest's 333k, one segment either way (lichen); 0.1.6 estimated the check at a few thousand. On a 4-vCPU x86 server it proves in 640–660 s and wraps in about 255 s (lichen).

5.5. Security, as a sketch

The statement cannot say that the hub signed b, only that a valid hub signature on b exists, and one exists for every b; only the hub can compute it. So what binds the amount is not that a false statement cannot be proved but that the prover must *know* a witness: that whoever is paid knows one, which then contains a hub signature, which unforgeability then forces to be on the asserted amount. The pipeline already relies on this: the chain proof's checks of users' signatures bind in the same way.

BABE gives the first step. Its knowledge soundness (Definition 9 of [5]) states that for any NP relation, a prover who gets the Success transaction onto the chain knows a witness, in the sense that an extractor recovers one from its view; it is proved for an honestly run setup (its Definition 16 and Theorem 8). Here the relation is the statement above, through two more layers: Groth16 is knowledge-sound (in the generic or algebraic group model), so the Groth16 proof yields a valid RISC Zero receipt; RISC Zero's STARKs are arguments of knowledge (in the random-oracle model), so the receipt yields the guest's input, (b, w, σ_H). If $b \neq b^*$, the asserted amount, then σ_H is a signature under pk_H on a message the hub never signed, and the hub signed exactly one: a forgery against a one-time signature.

What remains to be written properly:

- BABE proves knowledge soundness for an honestly run setup, not yet for one in which the prover takes part. Alice's part is small and every secret is the hub's (Section 3), so we expect the extension to be routine.
- The hub's signature arrives during the proving phase, after the statement is fixed, and depends on Alice's own assertion: the extraction must hold with the signature in the prover's view, which BABE's definition allows but which should be checked against its proof.
- The composition of extractors across BABE, Groth16 and the STARKs, under BABE's restriction to benign auxiliary input.

6. The transaction graph

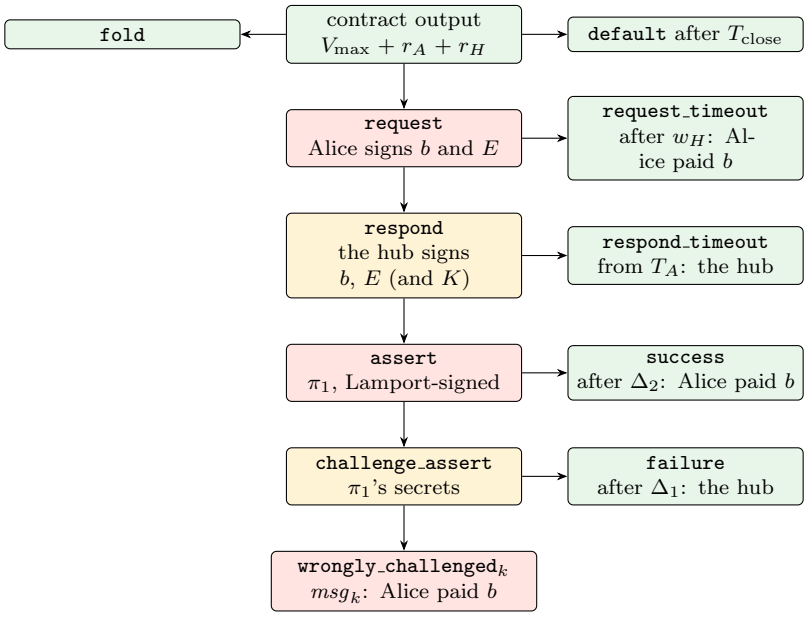


Figure 1: The dispute's graph as built. Red: Alice's links; yellow: the hub's; green: the terminals, each of which ends the dispute. One chain of links carries the contract's value, each link's output with its terminal; a terminal and the next link spend the same output, so only one of them confirms.

BABE's graph (its Appendix A.1) keeps the prover's deposit apart and has Failure spend three outputs, because there Assert starts from an outside coin. Here the dispute starts from the contract output, so the value flows through one chain and every transaction has one input (Figure 1, Table 1). A challenge and `success` spend the same output, so they cannot race, and BABE's condition $\Delta_2 > \Delta_1 + 2u$ is not needed. Each kept instance has its own msg_k, so there is one `wrongly_challenged` per kept instance. A terminal cannot pay into a pre-signed transaction, since its broadcaster's fee input changes its id, so every terminal that pays Alice carries the payout's bit outputs itself: `wrongly_challenged` is 0.82 kvB rather than BABE's 149 vB. The links carry a 330-sat anchor for their poster and pay a floor of their own, 1 sat/vB, out of the contract, so they relay alone.

All are pre-signed at opening: the default, four links, four terminals, and a `wrongly_challenged`

transaction	poster	spends, and what it carries or checks
`request`	Alice	the contract output; her one-time signature on b (one word); with the exit, on E too, and her claim posted
`request_timeout`	Alice	`request`'s output after w_H: the hub never responded; the payout
`respond`	hub	`request`'s output, within w_H; the hub's signature on b, and Alice's again for the digit-by-digit comparison; with the exit, the same for E, in one of two leaves: plain, or countering with K
`respond_timeout`	hub	`respond`'s output from the absolute T_A: Alice never asserted; the hub takes everything
`assert`	Alice	`respond`'s output, before T_A; π_1 under her Lamport key, in 2-bit digits, with π_2 and π_3 beside
`success`	Alice	`assert`'s output after Δ_2: no challenge; the payout
`challenge_assert`	hub	`assert`'s output, within Δ_2; the secrets of π_1's digits, by completing Alice's adaptor pre-signatures (Section 9)
`wrongly_challenged`$_k$	Alice	`challenge_assert`'s output, before `failure`; msg_k of kept instance k; the payout
`failure`	hub	`challenge_assert`'s output after Δ_1: no answer; the hub takes everything

Table 1: The transactions of the dispute, as built.

for each kept instance, 19 transactions at $(N, M) = (78, 10)$ (0.1.6 estimated about ten); with the exit, 20, since `respond`'s two leaves are each pre-signed. Each `wrongly_challenged` is signed by Alice alone and kept from the hub (Section 7). The payout and its bit outputs are as under BitVMX.

The assertion's deadline. Alice's heavy move, `assert`, needs no window: a late assertion only delays a payout the hub owes anyway, and the hub loses nothing by waiting. So its deadline T_A is absolute, T_{close} less Δ_2, Δ_1 and δ, so that the rest of the dispute and the payout fit, and the proof, which needs the hub's signature first, never races a timeout. On Mutinynet, with 30-second blocks, the windows are $w_H = 40$ (240 with the exit, Section 10), $\Delta_2 = 240$, $\Delta_1 = 80$ and $\delta = 40$ blocks, and a contract lasts a week, so an assertion is due a week after the contract opens. What remains due within a window is, for Alice, `wrongly_challenged` and, for the hub, its response and its challenge.

7. Alice's liveness: a keyless tower

Under BitVMX, Alice cannot delegate her moves without her keys. Each move signs, with a one-time key, hashes from her trace at positions the hub's last choice selects, so it cannot be signed in advance; a tower that plays her side holds her move keys, and by equivocating could lose her the contract [2, §10]. Making those moves keyless would need `OP_CAT`. BABE removes the problem rather than shrinking it: Alice's only commitments, b and π_1, are fixed before anything she must answer, and her one answer to the hub is a hash preimage, not a signature.

This is a difference of kind, not of count: a gadget with fewer interactive moves would still need someone holding her keys while the dispute runs; here no one does.

What she signs, and when. Two one-time signatures: on b, for `request`, made when she returns b on the L2; and on π_1, the Lamport signature of `assert`, made once she has the proof. The proof needs the hub's signature on b, so the second comes after `respond`, but before the loose, absolute deadline T_A: a second visit, not a race. A prover she does not trust with keys can make the proof; she checks it (one Groth16 verification) and signs π_1.

What the tower holds. Her signed `request`, her Lamport signature on π_1 (a preimage per digit, checked against her key), her signatures on the `wrongly_challenged`s, the proof, the contract's pre-signed transactions and her adaptor pre-signatures, the kept instances and the opened shares (about 240 MiB at $(78, 10)$), and the key of her fee budget, which pays the terminals' fees. From these and the chain it plays everything left to Alice:

- `assert`, from her signature on π_1 (`request` and `respond` are on chain already, since the proof needs the hub's signature);
- `request_timeout` and `success`: pre-signed, released by their timelocks;
- `wrongly_challenged`: it reads the secrets from `challenge_assert`, interpolates each kept instance's labels, evaluates the garbled circuits to $r\pi_1$, decrypts with π_2 and π_3, and posts the *msg* of the instance that opens.

Her bit outputs wait δ for her own key, as under BitVMX, and nothing races her for them: she sweeps them later from her own device, with the tower's state file.

On Mutinynet a tower played contract 1001's dispute this way, from lichen's Docker image on a rented 4-vCPU machine [4]: Alice requested, proved after the hub's response and exported the package (740 KB, and 243 MB of kept instances); the tower asserted and, the hub not challenging, posted `success` the block after Δ_2. Her device did nothing between the export and the sweep.

Two changes to BABE's graph, made. First, in BABE, the leaf `wrongly_challenged` spends checks the prover's signature as well as the hashlock, $\mathsf{HashLock}(h_{msg}) \wedge \mathsf{CheckSig}(pk_P)$, signed when posted (its Appendix A.1). A tower cannot sign then, but the signature should stay. Any spend of the leaf reveals *msg* and blocks `failure`, which is what Alice wants, but only once it confirms, and the hub knows *msg* from the start. Without the signature the hub could spend the leaf itself, in a large transaction at a low fee rate, which Alice's spend could replace only by paying more in all (BIP 125's third rule); if the hub's spend does not confirm within Δ_1, it replaces it with `failure`. So Alice makes the signature at setup instead: `wrongly_challenged` is fixed by then, and *msg* is in its witness, not under its signature. As every leaf in lichen's graph, it is 2-of-2, and she keeps her signature from the hub, for her towers only. She signs it `ALL|ANYONECANPAY`, as a TRUC (version 3) transaction, and the tower adds *msg* and a fee input of its own. A TRUC transaction cannot spend an unconfirmed parent of another version, so the tower's fee coin confirms first; `challenge_assert` has confirmed already, since Alice answers only a confirmed challenge. Bitcoin Core relays TRUC from version 28; on Mutinynet a node of an older build refused one with `version` while its newer peers relayed it, so lichen's wallet now

sends a transaction its node refuses for its version through the network's public Esplora.

Second, fees. The terminals (`request_timeout`, `success`, and `wrongly_challenged`) are signed `ALL|ANYONECANPAY`, and their broadcaster adds a coin of exactly the fee from Alice's fee budget, set aside when the contract opened [2, §6]: the tower holds that budget's key, which guards nothing else. (At 1 sat/vB a small terminal's fee is under the dust floor of a P2TR output, so the coin is at least 330 sats.) The links (`request`, `assert`) pay their floor themselves; above it they are bumped through an anchor only their poster can spend, which lichen still locks to the key that also sweeps her winnings. So a tower posts her links at their floor, which sufficed on Mutinynet; to bump them it needs the anchors to have a key of their own, guarding nothing but 330 sats each, which is not yet built.

A compromised tower can do little. It holds no key that signs a move, so it cannot sign a wrong move or a second value. That matters here: given her Lamport key, a tower that signed two values of π_1 would let the hub combine their bits into a third, signed digit by digit, and challenge with the secrets for it; she could not decrypt, and `failure` would take everything. So the tower is not trusted like her own machine, as under BitVMX, but like a Lightning watchtower: several can run at once, strangers to her and to each other, and any one suffices. At worst one stays silent, or spends her fee budget, 30,000 sats on Mutinynet. One caveat: a tower run by the hub holds the signed `wrongly_challenged`, and could post it with *msg* as soon as it posts `challenge_assert`, padded with inputs of its own, to pin it as above. TRUC's limit of 10 kvB on the transaction bounds what an honest tower must outbid, about 100,000 sats at 10 sat/vB. Her wallet, for its part, signs one π_1 only, and keeps the proof it signed.

8. Costs, measured

8.1. Transactions

Table 2 gives the sizes as mined on Mutinynet (lichen), or on regtest under mainnet's relay policy where marked, beside 0.1.6's estimates.

8.2. Disputes

Three disputes on Mutinynet (2026-10-29), each a withdrawal from a contract of cap $2^{21}-1$ sats, with a real proof [4] (Table 3). Every link went at its floor, 1 sat/vB, with no child: a dispute's fees are about its size in sats. Table 4 sets them against BitVMX's. Most of the gain against a vanished hub comes from not publishing the proof's input words before the hub moves; BitVMX could recover much of it by demanding them only from a hub that is present [2]. A false claim's dispute has one shape, whatever the lie: under BitVMX the liar chooses the lie, and with it the cost of the search that catches it, up to twice the cheapest.

8.3. Congestion

The main paper's rule, $Ns \leq w \cdot 1\,\mathrm{MvB}$ for N contracts whose move of s bytes is due in a window of w blocks, must hold for every window. Comparing gadgets by one window would be unfair to the one with fewer: a BitVMX dispute is twelve windowed moves on each side, one after another, so a hub safe at a given window under BitVMX asks for a dispute twelve windows long. The fair basis is the dispute's length T for a given N, with each window sized to its move:

transaction	measured	0.1.6
`request`: one signed word	0.45 kvB	∼0.4
`respond`: two signed words and the comparison	0.76 kvB	∼0.6
`assert`: π_1 in 254 2-bit digits	7.73 kvB	∼7.8
`challenge_assert`: 254 secrets by adaptor signatures	20.72 kvB (its leaf 60,232 bytes, 509 witness elements, regtest)	∼21
`wrongly_challenged`, with its fee input and the payout	0.82 kvB	∼0.21
`success`, with its fee input and the payout	0.78 kvB	∼0.3–0.8
`failure`, with its fee input	0.24 kvB	∼0.3–0.8
the hub's sweep of its bits; Alice's of hers	2.7; 1.7–1.8 kvB	∼3

Table 2: Transaction sizes, measured and as 0.1.6 estimated them.

ending	the dispute	what happened
a true claim, challenged	30.5 kvB in 5	the hub, hostile, challenged a valid proof; kept instance 9 of 10 opened; answered five blocks after the challenge
a true claim, unchallenged	9.7 kvB in 4	played by a tower in Docker; `success` the block after Δ_2
a false claim	29.9 kvB in 5	another contract's real proof; challenged, none of 10 kept instances opened; `failure`

Table 3: Three disputes on Mutinynet, 2026-10-29.

exit	BABE	BitVMX [2]
the hub present: a fold	0.2 kvB	0.2 kvB
the hub gone	∼1.2 kvB, `request` and its timeout (est. from the measured sizes; played on regtest)	23.1 kvB
the hub refuses a true claim	9.7 kvB in 4; 30.5 in 5 if it challenges	∼120 kvB in 23
the hub disproves a false claim	29.9 kvB in 5	100.4 kvB
the worst case	the same: there is no separate case for reads	200–212 kvB, a read lie
Alice's transactions, those due in a window	4 (request, assert, wrongly challenged, success), one	13, twelve
keys Alice's tower must hold	none that signs a move: the fee budget's	her move and hop keys, and her input words'
pre-signed per contract	19 (20 with the exit)	141

Table 4: BABE's exits, as measured, against BitVMX's [2].

$N \leq T \cdot 1\,\mathrm{MvB} / \sum_i s_i$, summed over the defender's windowed moves [2, §9]. Table 5 applies it to the measured sizes. Hostile users bind under BABE. For a dispute of the same length, BABE

	BABE, Alice against a hostile hub	BABE, the hub against hostile users	BitVMX, the binding side
windowed moves	`wrongly_challenged`, 0.82 kvB	`respond` 0.76 and `challenge_assert` 20.72	as built, Alice: twelve windows sized to her proof; with a window per move, the hub
$\sum_i s_i$	0.82 kvB	21.5 kvB	302 kvB; 121 kvB
contracts safe with a dispute of a day	∼175,000	6,700	480; 1,190
of a week	—	47,000	3,300; 8,350
the dispute for 60,000 contracts	49 blocks	8.9 days	126 days; 50 days

Table 5: Contracts a hub can hold before congesting the chain pays, by the dispute's length.

holds about 5.6 times as many contracts as BitVMX with a window per move, and 14 times as many as BitVMX as built. A hostile hub stops mattering, since Alice's only windowed move is 0.82 kvB. The hub's response, 0.76 kvB in w_H, is part of its sum: 60,000 contracts need w_H of at least 46 blocks. Chunking BitVMX's path into separate windows would not change these figures, since it shrinks the path's largest move but not its sum.

8.4. Reserves and time

A reserve must cover the other side's fees in a dispute it loses. The links pay their floors out of the contract, and the loser's reserve bears them: at most 30,981 sats at 1 sat/vB, the four links' floors and their anchors, against 197,356 under BitVMX (lichen). lichen's hub sizes its reserve to that figure, at the least. A hub that refuses a true claim and does not challenge loses only the first three links', 9,934; one that challenges pays for its own challenge too. At 10 sat/vB the deepest dispute's floors are about 310,000 sats, against about 1.2 million under BitVMX, so the contract suits balances from about a hundred thousand sats rather than half a million.

A hostile hub stretches a dispute to about $w_H + \Delta_2$ blocks after Alice's assertion, plus Δ_1 if it challenges. On Mutinynet the challenged true claim was answered 20 minutes after Alice requested, 15 of them her proving; the unchallenged one paid two hours after its assertion, that being Δ_2. A vanished hub's users wait w_H, as under BitVMX.

8.5. Setup per contract

The new cost, at lichen's $(N, M) = (78, 10)$ with verifiable secret sharing (Section 9), measured on one 14-core laptop (lichen), beside BABE's operating point with soldering (Table 6). Our

item	$(78, 10)$, VSSS (lichen)	$(2{,}268, 4)$, soldered (BABE)
an instance	23.83 MiB (gates 8.06, the encoding's tables 15.77); garbled in 1.2 s, evaluated in 0.6 s, one thread	22 MiB; 175 ms, 127 ms
the hub's setup	dealing 1.3 s, 78 instances 9.4 s	cut-and-choose 1 min 37 s for both roles, 1.8 GB at peak; soldering 745 s more
Alice's checks	1.5 + 8.1 s	(in the above)
in all	19 s (0.1.6 estimated 40)	∼14 min
what Alice downloads and stores	∼240 MiB, 10 instances	∼89 MiB, 4 instances
a hub opening 10,000 contracts	∼30 laptop-hours of the hub's own work	∼2,300 laptop-hours

Table 6: The setup per contract.

garbling hashes with BLAKE3 and multiplies schoolbook, where BABE's uses fixed-key AES and Karatsuba, hence the larger and slower instance. On Mutinynet the hub's setup took 9.5 s. The download is now the larger cost: about 240 MiB per contract, again at each renewal. Two ways to cut it remain open: Duty-Free Bits' 45-fold smaller encoding [9], unverified for these instances, and reusing the kept instances across renewals, sending a fresh *msg*, ciphertext and hashlock for each, since an unchallenged instance has revealed nothing.

9. Setup by verifiable secret sharing

Fourteen minutes of computation per contract, paid again at every renewal, is probably more than opening a contract can bear; twenty seconds is not. Glock [8] replaces soldering with

verifiable secret sharing (VSSS), and BABE's authors have prototyped that layer for BABE's 508 input bits (BABE, §9.4), without a transaction design. lichen builds it, with the transaction design below.

Why BABE keeps 4 of 2,268. Cut-and-choose fails only if every kept instance is bad and every opened one good, so the error is $\binom{N}{M}^{-1}$, held to 2^{-40}. Unsoldered, each kept instance puts a label set on chain; soldered, the proof's time grows with M (745, 866 and 969 s for 4, 5 and 6, BABE). Either way M must be small, and a small M forces a large N.

Secret sharing reverses that. The hub shares each input secret, for each digit and each value, across the N instances, with threshold $N - M + 1$, and publishes commitments to the polynomials (Feldman's, over secp256k1). Alice checks the opened instances' shares against them. The labels of a free-XOR garbling are related ($L^1 = L^0 \oplus \Delta$), so they cannot themselves be independent shares: the shared values are secrets of their own, which each instance maps to its labels through a small table, $H(s) \oplus L$. At proving time one secret per digit goes on chain, $f(0)$ of its polynomial; with the $N - M$ opened shares, Alice interpolates the secrets, and so the labels, of every kept instance. The value Alice did not assert stays hidden, since only $N - M$ points of its polynomial are known. The chain carries one secret set whatever M is, and the cost moves to N, the shares to commit and check. So the right choice is now a small N:

(N, M)	setup, both roles	Alice stores and receives
(2,268, 4), soldered	∼14 min (BABE)	∼89 MiB
(78, 10), VSSS	19 s (lichen)	∼240 MiB (lichen)
(124, 8), VSSS	∼1 min (BABE's estimate for the layer: 54 s)	∼190 MiB (est.)
(181, 7), VSSS	∼1.5 min (84 s for the layer)	∼170 MiB (est.)

$\binom{78}{10}^{-1}$ is $2^{-40.2}$; lichen's wallet refuses a setup of less than 2^{-40} off regtest. The bill moves to storage and transfer: Alice keeps every finalized instance, since she cannot know which one is honest. Glock's even split, about 44 instances with 22 kept, would be the wrong end of that trade here, at over 500 MiB.

On chain: adaptor signatures. The secret posted on chain must be the one Alice checked against the commitments, and the chain must check that. A hashlock cannot: tying a hash to a committed point sG needs a proof, which is what soldering is. Glock uses adaptor signatures instead. At setup Alice gives the hub, for each digit j and value β, a pre-signature on `challenge_assert` under her own one-time key $A_{j,\beta}$, adapted to the committed point $C_0 = f_{j,\beta}(0)\,G$. The script selects $A_{j,\beta}$ by her Lamport signature, which the hub copies from `assert`, and the hub can satisfy it only by completing her pre-signature, which hands her $f_{j,\beta}(0)$. The selection matters both ways: the other value's secret would leave Alice unable to decrypt, and both values' would let her evaluate the circuit on points other than π_1. In Glock the garbler is also the prover, so its adaptor signatures are its assertion; in BABE the prover asserts and the verifier reveals, so both are paid for. A completion for a value other than the asserted one does not verify (lichen).

Each secret is the adaptor of exactly one pre-signature, under a key used for nothing else.

Nothing is adapted twice to one point, so the setting is adaptor signatures' basic one-time case; one secret serves all M kept instances, but by interpolation off chain, not through M signatures. The pre-signatures are all on one sighash, under different keys, which BIP340's challenge includes. Alice makes them once the kept instances are chosen: their hashlocks are in `challenge_assert`'s output, and so in the sighash.

Two bits a step. Bit by bit, this does not fit in one input. Tapscript limits an input's initial stack to 1,000 elements (BIP 342), and each bit needs two, the hub's signature and Alice's preimage: 1,016 for π_1's 508 bits, 1,017 with the hub's own signature, which lichen's tests confirm fails where 1,000 passes. Alice therefore signs π_1 in 2-bit digits, revealing one of four independent preimages per digit: Lamport over a larger alphabet, which, unlike Winternitz's chains, needs no checksum. The hub shares one secret per digit and value, which each instance maps to both bits' labels. The polynomials number the same, $254 \times 4 = 508 \times 2$, as do the keys and hashes in the leaf, but the hub's signatures and Alice's preimages halve, to 254 each, and the stack holds 509 elements:

	bit by bit (est.)	2-bit digits (lichen)
`challenge_assert`'s stack	1,017 elements: two inputs	509
`challenge_assert`	∼26 kvB	20.72 kvB
`assert`	9.24 kvB (BABE)	7.73 kvB
a false claim disproved	∼37 kvB	29.9 kvB
a true claim challenged to the end	∼39 kvB	30.5 kvB
the hub's windowed sum	∼27 kvB	21.5 kvB
contracts for a week's dispute	37,000	47,000

Wider digits do not pay: at three bits the keys and hashes per bit rise from two to 8/3. The pre-signatures, 1,016, are a small transfer, about 100 KB.

The keyless tower survives. Alice's keys $A_{j,\beta}$ are used once, at setup, to make the pre-signatures, and never again. To read the secrets the tower needs her pre-signatures and the opened shares, none of them a key, and it extracts and interpolates as Alice would (Section 7).

10. The exit, with a counter

lichen's exit [1] lets Alice withdraw from a state already anchored on Bitcoin when the hub, which is also the sequencer, censors her return, stops sealing or vanishes. She claims her notes at an anchored state S; the hub may answer only with a *counter*: one of the notes spent at an anchored state after S, final before her claim, or her own signed claim of one of them through another contract. Version 0.1.7 of this note sketched two shapes; the first is built.

The claim, in `request`. Alice's request signs, beside b, the claim's fingerprint $E = H(c \,\|\, O_D \,\|\, b \,\|\, h_S \,\|\, SR_S \,\|\, nul$ under a second one-time key, and posts the claim in its witness: h_S, SR_S, each note's opening and its owner's signature on E, at most 1,169 bytes, in fifteen items of 80 bytes (Bitcoin Core relays no larger tapscript witness item). O_D is the contract's deposit coin, which only its funding

can spend, so it names the contract on Bitcoin and is known when the setup fixes the statement. The claim is posted for the hub to search, and to cite against her if she claims a note twice; a return's dispute signs a fixed E and posts nothing.

The counter, in `respond`. `respond` has two leaves, spending into one transaction, so that `assert` follows either. The plain leaf signs b, and E under a key e_{plain}; the countering leaf signs E under another key e_{counter}, and the counter K itself, a Groth16 proof of the counter guest compressed to 128 bytes (259 digits). Script can't tie a hash of K to signed digits without `OP_CAT`, so K is signed whole: compressed, since 256 bytes would exceed the 1,000-element stack limit at two elements a digit. Script checks the hub's E against Alice's by verifying the hub's signature tied to her signature's digits, without a separate comparison. On regtest under mainnet's relay policy (lichen): `request` 2.37 kvB with the largest claim; `respond` 4.48 kvB plain and 12.26 kvB countering, 766 witness elements.

The statement's three branches. The statement stays fixed at setup, with the journal $c \,\|\, \mathcal{D} \,\|\, H(pk_H \,\|\, O_D)$, pk_H all four of the hub's keys in `respond`. Alice wins on any of:

- *return:* the hub signed b, and a return of b to the contract is anchored, as before (the dispute of Section 5);
- *claim:* the hub signed b and E in the plain leaf, and her claim holds: her notes in S's tree and unspent there, S anchored with the contract's work on it, `request` in a block with that work on it too and posting exactly this claim, b at most their sum;
- *failed counter:* the hub signed E and K in the countering leaf, and K is not a counter for her $\mathcal{D} \,\|\, E$.

Which key signed E says which leaf ran; the hub's mode is a private input, like b, since it isn't known at setup. A true counter leaves her no branch, and `respond_timeout` takes the contract from T_A. The third branch verifies a Groth16 proof inside RISC Zero: 47M cycles with RISC Zero's accelerated substrate-bn (174M with arkworks), about 48 segments, so an hour or two on a laptop and minutes on a GPU. It needs no secret of Alice's, so anyone can prove it for her. The guest with the exit is 892 kB.

The hub's side. The counter guest proves either ground from the hub's ledger: a spend in a later anchored state whose history passes through S, its anchor confirmed in or before `request`'s block; or Alice's signed claim from another contract's `request`. The second matters because `respond` precedes `assert`: two requests in one block would otherwise both be paid, since no record of either exit exists yet. Proving a spend's counter needs the contract's work on `request`'s block first, then the proof and its wrap, so w_H becomes 240 blocks on Mutinynet (two hours) and 36 on signet. With no counter, the hub records the exit on the L2, with the owners' signatures, and the chain guest retires its notes.

What it adds. To every BABE contract: one more pre-signed transaction; about 1.6 kvB more in `request` for a return's dispute and about 3.7 kvB in `respond` (est., from the regtest sizes), so a refused true claim is about 15 kvB rather than 9.7, and a vanished hub's exit about 2.9 kvB rather than 1.2. The hub's windowed sum grows from 21.5 to about 25.2 kvB, or 33 kvB

if every response counters, so a dispute of a week holds about 40,000 contracts (30,500) rather than 47,000. Alice's windowed move is unchanged.

Limits. Shielded notes prove their owner's authority by knowing a key inside the proof, with nothing to post, so a shielded claim can't be cited and a wallet exits its transparent notes; shielded addresses will gain an owner key. A tower can't play an exit keylessly: Alice signs π_1 ahead for a return's dispute, but an exit's proof depends on which leaf the hub responds with. And the exit is safe exactly when Alice can read the anchored data she needs, which the main paper discusses [1].

11. Open questions

- The security argument of Section 5.5, written as a proof.
- A proof of robustness under cut-and-choose, against a hub that garbles wrongly. It is BABE's to give, but lichen's users rely on it.
- The security argument for the secret-sharing setup, with the shares and adaptor signatures in place of soldering.
- What Alice must download, about 240 MiB a contract: Duty-Free Bits' smaller encoding, and the kept instances' reuse across renewals.
- The anchors' own key, so that a tower can bump Alice's links above their floor.
- A tower for the exit: Alice's proof depends on the hub's response, so she can't sign it ahead.
- The exit played with real proofs: a counter proved and wrapped within w_H, and the failed-counter branch proved.
- Whether BABE's authors, or GOAT's, see a problem with moving the amount into the witness this way.

References

[1] waxwing. Lichen: L2s in channels enforced by Bitcoin. Version 0.3, October 2026. https://github.com/AdamISZ/lichen/blob/master/papers/lichen-v0.3.1.pdf

[2] waxwing. Lichen over BitVMX: the dispute, as built and measured. Version 0.1, October 2026. https://github.com/AdamISZ/lichen

[3] lichen: the ledger, the sequencer, the hub and the wallet. https://github.com/AdamISZ/lichen

[4] lichen's explainers: a BABE dispute, a true claim challenged; BABE's other endings. https://github.com/AdamISZ/lichen/tree/master/docs/explainers

[5] S. Garg, D. Kolonelos, M. Sergeevitch, S. Sridhar and D. Tse. BABE: verifying proofs on Bitcoin made 1000x cheaper. ePrint 2026/065, revised October 2026. https://eprint.iacr.org/2026/065

[6] S. Garg, D. Kolonelos, G.-V. Policharla and M. Wang. Threshold encryption with silent setup. CRYPTO 2024.

[7] L. Eagen and Y. T. Lai. Argo MAC: garbling with elliptic curve MACs. ePrint 2026/049. https://eprint.iacr.org/2026/049

[8] L. Eagen. Glock: garbled locks for Bitcoin. ePrint 2025/1485. https://eprint.iacr.org/2025/1485

[9] A. Khambhati, S. Bhattacharya and D. Heath. Duty-free bits. ePrint 2026/476. https://eprint.iacr.org/2026/476

[10] GOAT Network research. Dynamic public inputs in BABE (deferred binding). HackMD, 2026. https://hackmd.io/@goatresearch/B1N9g2uOee

[11] waxwing. Variable payout amounts for BABE, as applied to Lichen. Delving Bitcoin, October 2026. https://delvingbitcoin.org/t/variable-payout-amounts-for-babe-as-applied-to-lichen/2963

[12] S. D. Lerner, R. Amela, S. Mishra, M. Jonas and J. Álvarez Cid-Fuentes. BitVMX: a CPU for universal computation on Bitcoin, 2024. https://arxiv.org/abs/2405.06842

[13] RISC Zero. The RISC Zero zkVM and its Groth16 receipts. https://dev.risczero.com